

DATABASE MANAGEMENT SYSTEMS

Unit-wise Notes — Based on JNTUH (R22) B.Tech Syllabus

Unit	Title
I	Introduction to DBMS & the Entity-Relationship Model
II	Relational Model, Relational Algebra & Relational Calculus
III	SQL — Queries, Constraints, Triggers & Schema Refinement (Normalization)
IV	Transaction Management & Concurrency Control
V	Storage, File Organization & Indexing

These notes follow the standard JNTUH R22/R18 regulation syllabus for Database Management Systems (as taught in II Year B.Tech, CSE/IT branches), and are organized unit-wise for quick revision and exam preparation. Each unit covers definitions, core concepts, diagrams-in-text, and short/long answer style explanations.

UNIT I: Introduction to DBMS & the E-R Model

1.1 What is a Database and a DBMS?

A **database** is an organized, persistent collection of related data that models some aspect of the real world (e.g., a university's students, courses, and enrollments). A **Database Management System (DBMS)** is software that allows users to define, create, maintain, and control access to the database. Together, the database and the DBMS software (along with the applications built on top) are called a **database system**.

1.2 File System vs DBMS

Aspect	File System	DBMS
Redundancy	High — same data repeated across files	Minimized through normalization
Data Integrity	Difficult to enforce	Enforced via constraints
Concurrent Access	Not safely supported	Supported using concurrency control
Security	Limited, file-level only	Fine-grained, user/role based
Data Independence	Not present (programs tied to file structure)	Present (logical & physical)
Backup/Recovery	Manual, error-prone	Automatic and robust

1.3 Levels of Abstraction (Three-Schema Architecture)

- **Physical level:** lowest level; describes how data is actually stored (files, blocks, indexes).
- **Logical (conceptual) level:** describes what data is stored and the relationships among it, independent of physical storage.
- **View level:** highest level; describes only a part of the database relevant to a particular user/application.

Data independence is the ability to change the schema at one level without affecting the schema at the next higher level. **Physical data independence** = changing storage/physical structure without altering the logical schema. **Logical data independence** = changing the logical schema without altering application programs (views).

1.4 Instances and Schemas

The **schema** is the overall design/structure of the database (like a variable's type) — it rarely changes. An **instance** is the actual data stored in the database at a particular moment (like a variable's value) — it changes frequently as data is inserted, updated, or deleted.

1.5 Structure / Components of a DBMS

- **Storage Manager:** manages interaction with the OS file system — includes the authorization & integrity manager, transaction manager, file manager, and buffer manager.
- **Query Processor:** includes the DDL interpreter, DML compiler, and query evaluation engine that parses, optimizes, and executes queries.
- **Transaction Manager:** ensures the database remains consistent despite failures, and concurrent transactions proceed without conflicts.

1.6 Database Languages

- **DDL (Data Definition Language):** CREATE, ALTER, DROP — defines schema.
- **DML (Data Manipulation Language):** SELECT, INSERT, UPDATE, DELETE — manipulates data.

- **DCL (Data Control Language):** GRANT, REVOKE — controls access.
- **TCL (Transaction Control Language):** COMMIT, ROLLBACK, SAVEPOINT — manages transactions.

1.7 Database Users and Administrators

- **Naive users** — interact via pre-built application interfaces (e.g., ATM users).
- **Application programmers** — write application code using DML/host languages.
- **Sophisticated users** — write queries directly using a query language.
- **DBA (Database Administrator)** — has central control: schema definition, storage structure, authorization, and routine maintenance.

1.8 The Entity-Relationship (E-R) Model

The E-R model is a high-level conceptual data model used for database design. It describes data in terms of entities, attributes, and relationships, typically depicted using an **E-R diagram**.

1.9 Entities, Attributes, and Entity Sets

- **Entity:** a distinguishable real-world object/concept (e.g., a specific student, a specific course).
- **Entity Set:** a collection of similar entities (e.g., the set of all Students).
- **Attribute:** a property that describes an entity (e.g., Name, Roll_No).
- **Simple vs Composite attribute:** atomic (Age) vs made of sub-parts (Name → First_Name, Last_Name).
- **Single-valued vs Multi-valued attribute:** Age (single) vs Phone_Numbers (multiple).
- **Derived attribute:** computed from other attributes (Age derived from Date_of_Birth).
- **Key attribute:** uniquely identifies an entity within its entity set.

1.10 Relationships and Relationship Sets

A **relationship** is an association among two or more entities (e.g., a Student *enrolls in* a Course). A **relationship set** is a collection of similar relationships. The number of entity sets participating in a relationship is called its **degree** (binary, ternary, etc.).

1.11 Mapping Cardinalities (Relationship Cardinality Ratios)

- **One-to-One (1:1):** e.g., a Person has one Passport, a Passport belongs to one Person.
- **One-to-Many (1:N):** e.g., one Department has many Employees.
- **Many-to-One (N:1):** e.g., many Students belong to one Department.
- **Many-to-Many (M:N):** e.g., Students enroll in many Courses, and each Course has many Students.

1.12 Participation Constraints

- **Total participation:** every entity in the set must participate in at least one relationship (shown as a double line in E-R diagrams).
- **Partial participation:** only some entities need participate (single line).

1.13 Keys

- **Super Key:** any set of attributes that uniquely identifies an entity.
- **Candidate Key:** a minimal super key (no redundant attribute).
- **Primary Key:** the candidate key chosen by the designer to uniquely identify tuples.
- **Alternate Key:** candidate keys not chosen as the primary key.
- **Foreign Key:** an attribute in one relation that references the primary key of another relation, used to maintain referential integrity.

1.14 Additional / Advanced E-R Features

- **Weak Entity Set:** an entity set that does not have enough attributes to form a primary key on its own; it depends on a strong (owner/identifying) entity via an **identifying relationship**, and uses a **partial key (discriminator)** combined with the owner's key.
- **Generalization:** bottom-up process of combining lower-level entity sets into a higher-level entity set (e.g., Car and Truck generalize into Vehicle).
- **Specialization:** top-down process of defining subgroups (subclasses) within an entity set (e.g., Employee specializes into Manager and Engineer).
- **Aggregation:** treats a relationship set as a higher-level entity, allowing relationships between relationships.

1.15 Conceptual Design With the E-R Model — Design Issues

- Deciding whether to use an attribute or an entity set to represent an object.
- Deciding whether a real-world concept is best expressed as an entity set or a relationship set.
- Deciding on the use of ternary vs. multiple binary relationships.
- Placement of relationship attributes, and choosing appropriate cardinality/participation constraints.

*Note (exam tip): Unit 1 questions commonly ask you to **draw the E-R diagram** for a given scenario (e.g., University, Banking, Library system) — practice identifying entities, their attributes, relationships, cardinalities and participation constraints for 2-3 standard case studies.*

UNIT II: Relational Model, Relational Algebra & Calculus

2.1 The Relational Model — Basic Structure

- **Relation (Table):** a set of tuples; corresponds to an entity set or relationship set.
- **Tuple:** a single row of a relation, representing one record.
- **Attribute (Column):** a named field with a defined domain.
- **Domain:** the set of allowable values for an attribute.
- **Degree:** number of attributes in a relation. **Cardinality:** number of tuples in a relation.
- **Relation Schema:** the structure, e.g., Student(Roll_No, Name, Dept, Age).

2.2 Integrity Constraints Over Relations

- **Domain constraint:** every attribute value must be from its defined domain (or NULL if allowed).
- **Key constraint:** no two tuples can have the same value for the primary key (uniqueness + not null).
- **Entity integrity constraint:** primary key attributes cannot contain NULL values.
- **Referential integrity constraint:** a foreign key value must either match a primary key value in the referenced relation or be NULL — ensures consistency across relations.
- **General/Assertion constraints:** arbitrary conditions that must always hold (implemented via CHECK/ASSERTIONS).

2.3 Enforcing Integrity Constraints

Constraints are specified at schema/table creation time (using PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK clauses) and enforced by the DBMS on every INSERT, UPDATE, or DELETE. On a referential integrity violation, the DBMS can reject the operation, or apply an action such as CASCADE, SET NULL, or SET DEFAULT.

2.4 Logical Database Design & Views

Logical database design is the process of translating a conceptual (E-R) design into a relational schema (tables, keys, constraints). A **view** is a virtual table derived from one or more base tables using a query; it is not physically stored but computed when referenced. Views support data independence and security by restricting what a user can see. Tables and views can be destroyed with DROP TABLE / DROP VIEW, and modified with ALTER TABLE.

2.5 Relational Algebra

Relational Algebra is a **procedural** query language: operations take one or two relations as input and produce a new relation as output. Core operators:

Operator	Symbol	Description
Select	σ (sigma)	Selects tuples satisfying a given predicate: $\sigma(\text{condition})(R)$
Project	π (pi)	Selects specified columns, removing duplicates: $\pi(\text{attributes})(R)$
Union	$\cup$	Tuples in R or S (R, S must be union-compatible)
Set Difference	$-$	Tuples in R but not in S
Intersection	$\cap$	Tuples in both R and S
Cartesian Product	$\times$	Combines every tuple of R with every tuple of S
Rename	ρ (rho)	Renames a relation and/or its attributes

Join (Natural)	■	Combines related tuples from two relations over common attributes
----------------	---	---

Example: find names of students in the 'CSE' department:

```
 $\pi$  Name (  $\sigma$  Dept = 'CSE' (Student) )
```

Additional (derived) operators:

- **Theta Join (■ θ):** Cartesian product followed by a selection on condition θ .
- **Equi-join:** a theta join where θ uses only equality comparisons.
- **Natural Join:** equi-join on all common attributes, with duplicate columns removed automatically.
- **Division ($\div$):** used for queries of the form 'find X that are related to ALL Y' (e.g., students who have enrolled in every course offered).

2.6 Tuple Relational Calculus (TRC)

TRC is a **declarative/non-procedural** query language — you specify *what* to retrieve, not *how*. A TRC query has the form $\{ t \mid P(t) \}$ — the set of all tuples t such that predicate $P(t)$ is true. It uses quantifiers $\exists$ (there exists) and $\forall$ (for all).

Example: find names of students in 'CSE' department:

```
{ t.Name | Student(t) AND t.Dept = 'CSE' }
```

2.7 Domain Relational Calculus (DRC)

DRC is similar to TRC but the variables range over **single field domain values** instead of whole tuples. A DRC query has the form $\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$.

Example:

```
{ |  $\exists r, d, a$  ( Student(r, n, d, a) AND d = 'CSE' ) }
```

*Both TRC and DRC are equivalent in expressive power to relational algebra (this equivalence is called **relational completeness**) — any query expressible in one can be expressed in the others.*

UNIT III: SQL & Schema Refinement (Normalization)

3.1 Basic Form of an SQL Query

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition  
GROUP BY column  
HAVING group_condition  
ORDER BY column;
```

Execution (logical) order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

3.2 Set Operations: UNION, INTERSECT, EXCEPT

- **UNION:** combines results of two queries, removing duplicates (UNION ALL keeps duplicates).
- **INTERSECT:** returns rows common to both queries.
- **EXCEPT (MINUS in Oracle):** returns rows in the first query but not in the second.

```
SELECT City FROM Customers  
UNION  
SELECT City FROM Suppliers;
```

3.3 Nested and Correlated Subqueries

A **nested subquery** is a query inside another query, often used with IN, ANY, ALL, EXISTS. A **correlated subquery** references a column from the outer query, so it is re-evaluated once per outer row.

```
-- Correlated subquery example  
SELECT e1.Name FROM Employee e1  
WHERE e1.Salary > (  
    SELECT AVG(e2.Salary) FROM Employee e2  
    WHERE e2.Dept = e1.Dept  
);
```

3.4 Aggregate Functions, GROUP BY, HAVING

- Aggregate functions: **COUNT, SUM, AVG, MIN, MAX.**
- GROUP BY groups rows sharing values in specified columns so aggregates apply per group.
- HAVING filters groups (used after aggregation), unlike WHERE which filters rows before aggregation.

```
SELECT Dept, AVG(Salary) FROM Employee  
GROUP BY Dept  
HAVING AVG(Salary) > 50000;
```

3.5 NULL Values

NULL represents an unknown or missing value. Any arithmetic operation involving NULL yields NULL. Comparisons with NULL use three-valued logic (TRUE, FALSE, UNKNOWN) — use IS NULL / IS NOT NULL rather than '= NULL'.

3.6 Complex Integrity Constraints in SQL

- **NOT NULL, UNIQUE, CHECK** — column-level constraints.
- **PRIMARY KEY, FOREIGN KEY (REFERENCES)** — enforce entity and referential integrity.
- **ASSERTIONS** — constraints not tied to one table, checked across the whole database (rarely supported natively, usually emulated with triggers).

3.7 Views, Triggers and Active Databases

A **view** can be defined, altered and dropped: `CREATE VIEW ... AS SELECT ...`; some views are updatable if they map directly to a single base table without aggregation. A **trigger** is a stored procedure automatically executed (fired) in response to INSERT, UPDATE, or DELETE events — enabling **active database** behavior (e.g., automatically maintaining audit logs or enforcing complex business rules).

```
CREATE TRIGGER trg_after_insert
AFTER INSERT ON Orders
FOR EACH ROW
BEGIN
    UPDATE Inventory
    SET Stock = Stock - NEW.Qty
    WHERE ProductID = NEW.ProductID;
END;
```

3.8 Schema Refinement — Why Normalize?

Poorly designed relations suffer from **redundancy**, which in turn causes anomalies:

- **Insertion anomaly:** unable to add data about one entity without also having data about another.
- **Update anomaly:** a change must be made in multiple rows, risking inconsistency if missed anywhere.
- **Deletion anomaly:** deleting a row unintentionally removes other useful information.

3.9 Functional Dependencies (FDs)

A functional dependency $X \rightarrow Y$ means: for any two tuples that agree on attribute(s) X , they must also agree on Y (X *functionally determines* Y). FDs formalize the notion of a 'good' key and are the basis for normalization.

Armstrong's Axioms (sound and complete inference rules for FDs):

- **Reflexivity:** if $Y \subseteq X$, then $X \rightarrow Y$.
- **Augmentation:** if $X \rightarrow Y$, then $XZ \rightarrow YZ$.
- **Transitivity:** if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

Derived rules: Union, Decomposition, and Pseudo-transitivity — all can be proven from the three axioms above.

3.10 Normal Forms

Normal Form	Condition
1NF	Every attribute contains only atomic (indivisible) values — no repeating groups or multi-valued attributes.
2NF	Must be in 1NF, and every non-key attribute must be fully functionally dependent on the ENTIRE primary key (no partial dependencies).
3NF	Must be in 2NF, and no non-key attribute is transitively dependent on the primary key (no non-key attribute depends on another non-key attribute).
BCNF	For every non-trivial FD $X \rightarrow Y$, X must be a super key (a stronger form of 3NF, eliminates remaining anomalies).
4NF	Must be in BCNF, and have no non-trivial multi-valued dependency (MVD) other than a dependency from a super key.

3.11 Decomposition Properties

- **Lossless-join decomposition:** joining the decomposed relations back together (natural join) must reproduce exactly the original relation, with no spurious tuples.
- **Dependency-preserving decomposition:** every FD from the original relation can still be enforced without needing to join relations back together.

- **Multi-valued Dependency (MVD):** $X \twoheadrightarrow Y$ holds if, for a given X value, the set of associated Y values is independent of other attributes — leads to 4NF when handled.

Note (exam tip): Practice normalizing a given unnormalized relation step-by-step ($1NF \rightarrow 2NF \rightarrow 3NF \rightarrow BCNF$) with a worked example, and be ready to check whether a given decomposition is lossless using the standard tableau/attribute-closure method.

UNIT IV: Transaction Management & Concurrency Control

4.1 What is a Transaction?

A **transaction** is a unit of program execution that accesses and possibly updates data items, and must be executed atomically — either fully completed or fully rolled back.

4.2 ACID Properties

Property	Meaning
Atomicity	All operations of a transaction complete, or none do (all-or-nothing).
Consistency	A transaction takes the database from one consistent state to another.
Isolation	Concurrent transactions do not interfere; effect is as if executed serially.
Durability	Once committed, changes persist even after a system failure.

4.3 Transaction States

- **Active** → **Partially Committed** → **Committed** (success path), or
- **Active** → **Failed** → **Aborted** (failure path, followed by rollback and possible restart).

4.4 Concurrent Execution & Schedules

A **schedule** is an ordering of operations from multiple transactions. Interleaving improves throughput and resource utilization, but must be controlled to avoid inconsistency.

- **Serial schedule:** transactions execute one completely after another — always consistent, but no concurrency.
- **Serializable schedule:** a (possibly interleaved) schedule that produces the same result as some serial schedule — the goal of concurrency control.
- **Conflict serializability:** tested by checking whether swapping non-conflicting adjacent operations can transform the schedule into a serial one (verified using a **precedence/conflict graph** — if it is acyclic, the schedule is conflict serializable).
- **View serializability:** a more general (less strict) notion based on read/write views rather than conflicts.

4.5 Recoverable and Cascadeless Schedules

- **Recoverable schedule:** a transaction commits only after all transactions whose changes it read have already committed.
- **Cascadeless schedule:** a transaction may read a value only after the writing transaction has committed — avoids cascading rollbacks.

4.6 Concurrency Control Protocols

(a) Lock-Based Protocols

- **Shared lock (S):** allows read access; multiple transactions can hold S-locks on the same item simultaneously.
- **Exclusive lock (X):** allows read+write; only one transaction may hold an X-lock, and no other lock is allowed on that item at the same time.

- **Two-Phase Locking (2PL):** every transaction has a *growing phase* (acquiring locks) and a *shrinking phase* (releasing locks), with no new locks acquired once the first lock is released — guarantees conflict serializability.
- **Strict 2PL:** all exclusive locks are held until the transaction commits/aborts — ensures cascadeless (and recoverable) schedules; widely used in practice.

(b) Timestamp-Based Protocols

Each transaction gets a unique timestamp at start. Conflicting operations are ordered by timestamp; if a transaction tries to read/write a value that violates timestamp order, it is rolled back and restarted with a new timestamp — guarantees serializability equivalent to the timestamp order, without using locks.

(c) Validation-Based (Optimistic) Protocol

Assumes conflicts are rare. Transactions execute in three phases: **Read phase** (work on local copies), **Validation phase** (check for conflicts against other transactions), and **Write phase** (apply changes if validation passes). Suited to workloads with low contention.

4.7 Deadlock Handling

- **Deadlock prevention:** ordering transactions using timestamps (e.g., Wait-Die, Wound-Wait schemes).
- **Deadlock detection:** build a wait-for graph; a cycle indicates deadlock, resolved by aborting a victim transaction.
- **Deadlock avoidance:** use resource-ordering / lock ordering to prevent circular wait conditions.

4.8 Failure Classification

- **Transaction failure:** logical errors or system errors causing a transaction to abort.
- **System crash:** hardware/software failure causing loss of volatile (main memory) storage.
- **Disk failure:** loss of data on non-volatile storage (disk block corruption or media failure).

4.9 Log-Based Recovery

The DBMS maintains a **log** — a sequential record of all update actions — on stable storage. Each log record typically stores the transaction ID, data item, old value, and new value. On recovery after a crash, the system uses:

- **REDO:** reapply changes of committed transactions found in the log (ensures durability).
- **UNDO:** reverse changes of transactions that had not committed at the time of the crash (ensures atomicity).

4.10 Checkpoints

A **checkpoint** is a point where the DBMS writes all buffered log records and modified data to stable storage, and records this fact in the log. During recovery, the system only needs to consider transactions active since the most recent checkpoint, greatly reducing recovery time by avoiding a full log scan.

4.11 Two-Phase Commit Protocol (2PC)

Used in distributed databases to ensure atomicity of a transaction spanning multiple sites. **Phase 1 (voting/prepare):** the coordinator asks all participants if they are ready to commit; each replies YES (ready) or NO. **Phase 2 (decision):** if all voted YES, coordinator sends COMMIT to all; if any voted NO, it sends ABORT to all — guarantees all sites reach the same decision.

UNIT V: Storage, File Organization & Indexing

5.1 Data on External Storage

Databases store data on secondary storage (disks) since it is non-volatile and offers larger capacity than main memory. Data is organized into **files**, and each file into fixed-size **blocks/pages** that are the unit of I/O transfer between disk and memory.

5.2 RAID (Redundant Array of Independent Disks)

RAID uses multiple disks in combination to improve performance (via striping/parallelism) and reliability (via redundancy/mirroring or parity). Common levels include RAID 0 (striping, no redundancy), RAID 1 (mirroring), and RAID 5 (block-level striping with distributed parity).

5.3 File Organization — Organizing Records in Files

- **Heap (unordered) file organization:** records placed in no particular order; fast insertion, but scanning for a search requires a full file scan.
- **Sorted (sequential) file organization:** records are kept sorted by a search key; supports efficient range queries but insertion/deletion is costly (may require reorganizing/shifting records).
- **Hashed file organization:** a hash function maps the search key to a specific block/bucket; very fast equality search, but poor for range queries.

File Organization	Search Cost	Insert Cost	Best For
Heap	High (full scan)	Low	Bulk loading, small files
Sorted	Low (binary search)	High	Range queries
Hashed	Very low (equality)	Low (avg)	Exact-match lookups

5.4 Indexing — Basics

An **index** is an auxiliary data structure built on one or more fields of a file to speed up retrieval of records satisfying a given search condition, at the cost of extra storage and maintenance overhead on updates.

5.5 Primary, Secondary and Clustering Indexes

- **Primary index:** built on the ordering (sort) key of a sequentially ordered file — usually the primary key.
- **Clustering index:** built on a non-key field by which the file is physically ordered — records with the same value are stored together.
- **Secondary index:** built on a non-ordering field; provides an access path independent of the physical order of the file, at the cost of extra space (there can be many secondary indexes on one file).
- **Dense index:** has an index entry for every search-key value (record) in the file.
- **Sparse index:** has index entries only for some records (e.g., one per block) — saves space, needs the file to be sorted.

5.6 Index Data Structures

(a) Hash-Based Indexing

- **Static hashing:** a fixed number of buckets; a hash function maps search-key values to bucket addresses — suffers from bucket overflow as the file grows.

- **Extendable (extendible) hashing:** dynamically doubles/halves the number of buckets using a directory and global/local depth, adapting gracefully to growth without full re-hashing.
- **Linear hashing:** grows the file one bucket at a time in a round-robin manner, avoiding the need for a directory structure altogether.

(b) Tree-Based Indexing

Tree indexes are ideal for both equality and range queries. The **intuition** is analogous to a multi-level 'phone-book' style structure that lets you narrow down the search range quickly at each level.

- **ISAM (Indexed Sequential Access Method):** a static, multi-level tree structure — the index levels are built once and remain fixed; overflow chains are used for insertions beyond the pre-allocated space, which can degrade performance over time.
- **B+ Tree:** a balanced, dynamic (self-adjusting) multi-level tree — the most widely used index structure in commercial DBMSs.

5.7 B+ Trees — A Dynamic Index Structure

- All data entries reside only in the **leaf nodes**, which are linked together in a sorted linked list — enabling efficient range queries by simply scanning across leaves.
- Internal (non-leaf) nodes only guide the search and contain no actual data — just key values and child pointers.
- Every leaf is at the **same depth**, keeping the tree perfectly height-balanced and giving uniform (logarithmic) search cost.
- Each node (except the root) is kept at least half full, which is maintained automatically through **splitting** (on insertion/overflow) and **merging/redistribution** (on deletion/underflow).

Search in a B+ Tree:

Start at the root; at each internal node, compare the search key against the node's key values to choose the correct child pointer, and recurse downward until a leaf is reached — the leaf directly contains the matching data entry (or a pointer to the actual record).

Insertion in a B+ Tree:

Find the correct leaf and insert the new entry in sorted order. If the leaf overflows (exceeds capacity), **split** it into two leaves and push (copy up) the middle key to the parent. If the parent then overflows, the split propagates upward — possibly creating a new root and increasing the tree's height by one level.

Deletion in a B+ Tree:

Remove the entry from the leaf. If the leaf underflows (falls below the minimum occupancy), first try to **redistribute (borrow)** an entry from a sibling; if that is not possible, **merge** the leaf with a sibling, and update (or remove) the corresponding key in the parent — this may propagate upward and can reduce the tree's height.

5.8 Comparing File Organizations (I/O Cost Summary)

Operation	Heap	Sorted	B+ Tree (Clustered)
Scan all records	Low	Low	Low
Equality search	High	Medium (binary search)	Low (log-based)
Range search	High	Low	Low
Insert	Low	High	Low (amortized)

Delete	Medium	High	Low (amortized)
--------	--------	------	-----------------

Note (exam tip): Be ready to manually construct a B+ tree of a given order for a sequence of insertions, and show the resulting tree after a couple of deletions — this is one of the most frequently asked long-answer questions in Unit V.

Reference textbooks: *Database System Concepts* — Silberschatz, Korth, Sudarshan; *Database Management Systems* — Raghu Ramakrishnan, Johannes Gehrke.